

Hackathon — Problem Statements

Detailed problem statement document for the CredX-themed hiring hackathon (Angular & Spring Boot developers)

This document expands on the three problem statements from the hackathon plan with full detail — objectives, user roles, feature scope, suggested tech stack, and evaluation criteria — so teams have everything they need at kickoff. Teams pick one problem statement and build a working product (Angular frontend + Spring Boot backend + database) within the 24-hour coding round.

PS 1 | Smart Job Matching Dashboard

Overview

Build a platform where students create a detailed profile and get matched to relevant job and internship postings. The core challenge is designing a matching experience that feels genuinely useful — not just a filtered list, but a ranked, explainable set of recommendations.

Target Users

- Students — build a profile and discover relevant postings
- Recruiters / companies (optional, if time allows) — post roles that feed the matching engine

Core Features

- **Student profile creation** — skills (tagged, multi-select), resume upload, GPA, work authorization / visa sponsorship status
- **Job/internship listings** — seeded or admin-added postings with role, location, work mode, and sponsorship requirement
- **Matching engine** — compute a **match score** per listing based on skill overlap, GPA thresholds, and work-auth compatibility
- **Filters** — role, location, remote/onsite/hybrid, visa sponsorship needed
- **Match score indicator** — visible per listing card (e.g. percentage or star rating), ideally with a short breakdown of why it matched
- **Application tracking (stretch)** — let students mark a listing as applied and track status

Suggested Tech Stack

Frontend

Angular — reactive forms for profile creation, custom pipe for match-score formatting, route guards if adding auth

Backend

Spring Boot — REST APIs for profiles/listings/matching, a scoring service that runs the match logic

Database	Relational (MySQL/PostgreSQL) or document store — student profiles, listings, and computed match results
Matching Logic	Rule-based scoring is sufficient (weighted skill overlap + GPA + work-auth check); ML-based matching is a bonus, not required

Evaluation Criteria

- Correctness and sensibility of the match-score logic
- End-to-end functionality — profile creation to matched results, without manual steps
- Usability of filters and clarity of the match score in the UI
- Code quality and API design

Bonus / Stretch Goals

- Explain-the-match tooltip (e.g. "Matched on: React, SQL, GPA above threshold")
- Recruiter-side view to see which students match a posting
- Sorting listings by match score instead of just filtering

Note: This problem statement rewards clean, explainable logic over a fully polished UI — judges should be able to see why a given student matched a given listing.

PS 2 | Campus Recruiting Portal

Overview

Build a portal where companies post roles specifically for a university's students. Students apply to postings, and university placement cell admins approve or reject postings and view analytics on recruiting activity.

Target Users

- Companies — post roles targeted at the university
- Students — browse and apply to approved postings
- Placement cell admins — approve/reject postings, view analytics

Core Features

- **Company role posting** — company creates a posting with role details, eligibility criteria, and deadline
- **Admin approval workflow** — placement cell admin reviews new postings and approves or rejects them before they go live to students
- **Student application flow** — students view approved postings and apply directly through the portal
- **Analytics dashboard** — admin view showing applications per company and overall placement rate
- **Status tracking** — postings move through clear states (pending → approved/rejected → closed)

Suggested Tech Stack

Frontend	Angular — role-based views (company / student / admin), reactive forms for posting creation and applications
Backend	Spring Boot — REST APIs with role-based access control for the three user types
Database	Relational (MySQL/PostgreSQL) — companies, postings, applications, approval status, aggregate stats
Auth (optional)	Simple role-based login (company / student / admin) if time allows; can be mocked with a role selector for the hackathon

Evaluation Criteria

- Correctness of the approval workflow (postings shouldn't be visible to students until approved)
- Accuracy of analytics (applications per company, placement rate)
- Clarity of role separation between company, student, and admin views
- Code quality and data modeling

Bonus / Stretch Goals

- Email/notification-style alerts on approval or application status change (can be simulated in-app)

- Charts for analytics instead of raw numbers
- Deadline-based auto-closing of postings

Note: The approval workflow is the crux of this problem statement — teams should make sure a posting truly cannot reach students without admin approval